

Department of Computer Science
Ashoka University
Advanced Algorithms
Assignment 2

Collaborators: None

Name: Rushil Gupta

Problem 1

Tight Bounds on c_2 and c_3

Let c_d denote the maximum number points in a d -dimensional unit cube that are mutually separated by at least distance 1. We will calculate tight lower and upper bounds on c_2 and c_3 .

Upper Bound: Consider a d -dimensional unit cube. We can partition this cube into 2^d sub-cubes of side length $\frac{1}{2}$. Now, the maximum distance between any two points in any of these sub-cubes is $\sqrt{d} \cdot \frac{1}{2}$ which is less than 1 for $d = 2, 3$. Therefore, we can have at most one point in each of these sub-cubes to ensure that all points are mutually separated by at least distance 1. Hence, we have the upper bound $c_2 \leq 4$ and $c_3 \leq 8$.

Lower Bound: We can place points at each of the corners of the d -dimensional unit cube. The distance between any two corners of the cube is at least 1, since at least 1 coordinate will differ by 1. So, we can place one point at each corner of the cube. This gives us a lower bound of $c_2 \geq 4$ and $c_3 \geq 8$.

Since the lower and upper bounds match, it follows that they are tight, and a stronger conclusion can be drawn: $c_2 = 4$ and $c_3 = 8$.

An Efficient Data Structure for the RIC Algorithm

We will use hashing to design an efficient data structure for the RIC algorithm in 2 and 3 dimensions (the observation earlier allows us to analyse the algorithm better). To hash a point, say $p = (x, y)$, the algorithm first computes the cell in which the point lies, i.e. $c = (\lfloor \frac{x}{\delta} \rfloor, \lfloor \frac{y}{\delta} \rfloor)$, where δ is the cell size. This will be the key of the point in the hash table. The following analysis will now assume $d \in 2, 3$, and hence not always show the dependence on d explicitly.

Time to see if the closest pair distance has changed: When we insert a new point into the data structure, we need to check if it is closer than the current closest pair distance δ . To do this, we only need to check the points in the same cell and the neighboring cells. Since we have established that there can be at most c_d points in any cell, and there are 3^d neighboring cells (including the cell itself), the total number of points we need to check is at most $c_d \cdot 3^d$. This means for 2-d, we need to check at most $4 \cdot 9 = 36$ points, and for 3-d, we need to check at most $8 \cdot 27 = 216$ points. Therefore, the time complexity for checking if the closest pair distance has changed is $O(1)$, since c_d and 3^d are constants for fixed dimensions $d = 2, 3$.

Time to rebuild: When we need to rebuild the data structure (when the minimum distance δ changes), we can rehash all the points in the current data structure into a new hash table with the updated cell size. The time complexity for rebuilding the data structure is $O(n)$, where n is the number of points in the data structure. This is because we need to rehash each point once.

Problem 4

No Constraint on Customer-Annoyance Index

Consider the first part of the problem where we have to maximise the profit for the TV network without any constraint on the customer-annoyance index. This is a just 0/1 Knapsack problem reframed: the profits are the prices p_i , the weights are the time slots t_i , and the capacity of the knapsack is the total time T . We can use a greedy method to get a $\frac{1}{2}$ factor approximation for this problem, i.e., if O^* is the optimal solution, we can get a solution O such that $p(O) \geq \frac{1}{2}p(O^*)$.

With Constraint on Customer-Annoyance Index

Notation: The notation is a bit hand-wavy, but here is a definition to help:

$$\begin{aligned} p(x_i) &:= p_i x_i \quad x_i \in [0, 1] \\ p(x) &:= \sum_{i=1}^n p(x_i) \quad x \in [0, 1]^n \end{aligned}$$

Now, we consider the full problem where we have to maximise the profit while keeping the total customer-annoyance index within a limit L . This is like having 2 knapsack constraints. Writing this as an integer linear program, we have:

$$\begin{aligned} \max \quad & \sum_{i=1}^n p_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n t_i x_i \leq T \\ & \sum_{i=1}^n l_i x_i \leq L \end{aligned}$$

Like we did for the usual 0/1 knapsack problem, we can relax the integer constraints to get a linear program. Say the solution to this linear program is $\hat{x}$. Since we have 2 constraints, at most 2 variables in $\hat{x}$ will be fractional, and the rest will be either 0 or 1. Let x^* be the optimal integer solution. Then, make the following observation:

$$p(x^*) \leq p(\hat{x})$$

We now discuss the rounding scheme. Let $\bar{x}$ be the rounded solution where we take all the variables that are 1 in $\hat{x}$ and set them to 1 in $\bar{x}$, and set the rest to 0. Let the fractional variables in $\hat{x}$ be f_1 and f_2 . Then, observe that $\bar{x}$, $\{f_1\}$, and $\{f_2\}$ are all feasible solutions to the integer linear program. Then:

$$\begin{aligned} p(\hat{x}) &= p(\bar{x}) + p(f_1) + p(f_2) \\ &\leq 3 \cdot \max(p(\bar{x}), p(f_1), p(f_2)) \end{aligned}$$

So, the algorithm outputs the maximum of the 3 solutions $\bar{x}$, $\{f_1\}$, and $\{f_2\}$. Then, if $\tilde{x}$ is the output of the algorithm, we have:

$$p(\tilde{x}) \geq \frac{1}{3}p(x^*)$$

The running time of this algorithm is dependent on the running time of the linear program solver, which we know is polynomial in n . The rounding scheme takes linear time in n . Therefore, the overall running time of the algorithm is polynomial in n .

Problem 6

Let E be a set of n elements, and $\mathcal{F} = \{S_1, \dots, S_m\}$ be a family of subsets of E . The optimisation subset problem is then: find the min k such that there exists a subset of $\mathcal{F}$ of size k whose union is E . Now, given the fact that an element can belong to at most f sets in $\mathcal{F}$, the relaxed linear program is:

$$\begin{aligned} \min \quad & \sum_{i=1}^m x_i \\ \text{s.t.} \quad & \sum_{e \in S_i} x_i \geq 1 \quad \forall e \in E \\ & x_i \in [0, 1] \quad \forall i \in [1, m] \end{aligned}$$

Since each element belongs to at most f sets, $\sum_{e \in S_i} x_i$ is bounded by f . Then, say $\hat{x}$ is the optimal solution to the relaxed linear program. Then, let's look at the inequality constraint, and make the following observation for any element $e \in E$:

$$\begin{aligned} f &\geq \sum_{e \in S_i} \hat{x}_i \geq 1 \\ \Rightarrow f \cdot \max_{e \in S_i} \hat{x}_i &\geq 1 \\ \Rightarrow \max_{e \in S_i} \hat{x}_i &\geq \frac{1}{f} \end{aligned}$$

Consider the following rounding scheme: $\bar{x}_i = 1$ if $\hat{x}_i \geq \frac{1}{f}$, and $\bar{x}_i = 0$ otherwise. Then, from the above observation, we can see that for any element $e \in E$, there exists at least one set S_i such that $\bar{x}_i = 1$ and $e \in S_i$. Therefore, $\bar{x}$ is a feasible solution to the original optimisation subset problem.

We will now analyse the approximation factor of this rounding scheme. Let x^* be the optimal integer solution, and let $\bar{x}$ be the rounded solution. Then, we have:

$$\begin{aligned} \sum_{i=1}^m \bar{x}_i &\leq \sum_{i=1}^m f \cdot \hat{x}_i \quad (\text{definition of the rounding scheme}) \\ &= f \cdot \sum_{i=1}^m \hat{x}_i \\ &\leq f \cdot \sum_{i=1}^m x_i^* \quad (\text{since } \hat{x} \text{ is the solution to the relaxed LP}) \end{aligned}$$

Thus, the approximation factor of this rounding scheme is f . The running time of this algorithm is polynomial in n since solving the LP takes polynomial time, and the rounding scheme takes linear time in m .

Remark: I ended up swapping the meaning of $\bar{x}$ and $\hat{x}$ from the question's notation. However, I have defined them clearly here to avoid confusion. Apologies!

Problem 8

Part (a)

Consider the decision version of the problem: given m identical machines and n jobs with processing times $p_1, p_2, \dots, p_n$, is there an assignment of jobs to machines such that the completion time is at most k ? We will show that this problem is NP-Complete by reducing the Partition problem to it.

Partition Problem: Given a set of positive integers $S = \{s_1, s_2, \dots, s_n\}$, can we partition S into two subsets S_1 and S_2 such that the sum of the elements in S_1 is equal to the sum of the elements in S_2 ?

Reduction: Given an instance of the Partition problem with set S , we can construct an instance of the scheduling problem as follows: set the number of machines $m = 2$, the number of jobs $n = |S|$, and the processing times of the jobs $p_i = s_i$ for each i . Then, the decision question is: is there an assignment of jobs to the two machines such that the completion time is at most $k = \frac{1}{2} \sum_{i=1}^n s_i$?

Proof of equivalence: If there is a partition of S into two subsets S_1 and S_2 with equal sum, then we can assign the jobs corresponding to S_1 to machine 1 and the jobs corresponding to S_2 to machine 2. This will result in both machines having a completion time of $\frac{1}{2} \sum_{i=1}^n s_i$, which is what we want.

Conversely, if there is an assignment of jobs to the two machines such that the completion time is at most $k = \frac{1}{2} \sum_{i=1}^n s_i$, then the jobs assigned to machine 1 and machine 2 form two subsets of S with equal sum. Therefore, we have shown that the scheduling problem is NP-Complete.

Part (b)

Now, to get a simple lower bound on the completion time, we can use the following observation: any given job must be assigned to one machine, and will take p_i time on that machine. So, regardless of the assignment, since each job needs to be completed, the total processing time of all jobs must be greater than or equal to each job's processing time. Hence, a lower bound is $\max_i p_i$.

To derive another lower bound, consider the problem where each job can be split across multiple machines. In this case, the optimal assignment would be to distribute the jobs evenly across all machines. For this problem, the completion time would be:

$$\frac{1}{m} \sum_{i=1}^n p_i$$

Now, since allowing jobs to be split can only decrease the completion time, this is another lower bound on the completion time.

Part (c)

Let T be the time taken by the greedy algorithm, and T^* be the optimal completion time. Let j be the last job to finish in the greedy assignment. Then, it has processing time p_j and started at time $s = T - p_j$. Since the greedy algorithm assigns jobs to the machine with the least load, at time s , all machines must have load at least s . Therefore, the total time needed to process all the jobs scheduled before job j is at least $m \cdot s$. Thus, we have:

$$\begin{aligned} \sum_i p_i &\geq m \cdot s + p_j \\ \Rightarrow \sum_i p_i &\geq m \cdot (T - p_j) + p_j \\ \Rightarrow \sum_i p_i &\geq mT - (m - 1)p_j \\ \Rightarrow T &\leq \frac{1}{m} \sum_i p_i + \left(1 - \frac{1}{m}\right) p_j \end{aligned}$$

Now, using the lower bounds derived in part (b), we have:

$$T^* \geq \frac{1}{m} \sum_i p_i$$

$$T^* \geq p_j$$

Using these inequalities in the previous equation, we get:

$$T \leq T^* + \left(1 - \frac{1}{m}\right) T^*$$

$$\Rightarrow T \leq \left(2 - \frac{1}{m}\right) T^*$$

Thus, the greedy algorithm is a $(2 - \frac{1}{m})$ -approximation algorithm for the problem.

Part (d)

Consider the case where we have m^2 jobs with unit processing times and exactly one job with processing time m . The optimal assignment would be to assign the m length job and another unit length job to one machine, and distribute the remaining $m^2 - 1$ unit length jobs evenly across the other $m - 1$ machines. This would result in a completion time of $m + 1$. Moreover, this is optimal since $\frac{m^2+m}{m} = m + 1$, which is one of the lower bounds derived in part (b).

However, this algorithm has issues since it does not order the elements. If the m length job is assigned last, then the greedy algorithm would assign all the unit length jobs first, resulting in each machine having m unit length jobs. Then, when the m length job is assigned, it would be assigned to one of the machines with m unit length jobs, resulting in a completion time of $2m$.

Problem 9

Part (i) and (ii)

We will reduce the Hamiltonian Cycle problem to the TSP problem to show that it is NP-Hard. The Hamiltonian Cycle problem is defined as: given an undirected graph $G(V, E)$, does there exist a cycle that visits each vertex exactly once?

Let $G(V, E)$ be an instance of the Hamiltonian Cycle problem. We can construct an instance of the TSP problem as follows: create a complete graph $G'(V, V \times V, w)$ where the weight function w is defined as:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{otherwise} \end{cases}$$

Now, we will show that G has a Hamiltonian Cycle if and only if G' has a TSP tour of weight at most $|V|$.

($\Rightarrow$) Suppose G has a Hamiltonian Cycle C . Then, we can construct a TSP tour in G' by following the edges of C . The weight of this tour is exactly $|V|$, since each edge in C has weight 1.

($\Leftarrow$) Now, suppose G' has a TSP tour of weight at most $|V|$. Since the tour has weight $|V|$ and each edge in G' has weight at least 1, it follows that the tour must use only edges of weight 1. Therefore, the tour corresponds to a Hamiltonian Cycle in G .

Thus, we have shown that G has a Hamiltonian Cycle if and only if G' has a TSP tour of weight at most $|V|$.

Part (iii)

Suppose there exists a polynomial time approximation algorithm for TSP that can compute a tour within factor n of the optimal tour, where n is the number of vertices in the graph. In the reduction we did in part 1, we could have had any weights greater than 1 given to the edges not in E . So, let's set the weights of these edges to be $n^2 + 1$. Then, if there exists a Hamiltonian Cycle in G , the optimal TSP tour in G' would have weight n . If there does not exist a Hamiltonian Cycle in G , then any TSP tour in G' would have weight at least $n^2 + 1$.

Now, if we had a polynomial time approximation algorithm that could compute a TSP tour within factor n of the optimal tour, we could use it to solve the Hamiltonian Cycle problem as follows: run the approximation algorithm on G' . If the weight of the returned tour is at most n^2 , then we can conclude that there exists a Hamiltonian Cycle in G . If the weight of the returned tour is greater than n^2 , then we can conclude that there does not exist a Hamiltonian Cycle in G .

Hence, we cannot have a polynomial time approximation algorithm that can compute a TSP tour within factor n of the optimal tour, unless $P = NP$.